DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 3 – EF Core, migrations, seed, pooling, transações, concorrência e acesso leve

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** EF Core, migrations, seed, pooling, transações, concorrência e acesso leve	**Relatório 3**

Unidade 3 – Persistência e conexão com banco de dados

Objetivo: Persistir as tarefas em banco relacional, aplicar migrations e seed, tratar concorrência e comparar EF Core com Dapper ou ADO.NET.

1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); commit `614fc20`. Provedor **SQL Server 2022** em contêiner Docker (`mcr.microsoft.com/mssql/server:2022-latest`), **EF Core 10.0.14** (`Microsoft.EntityFrameworkCore.SqlServer` e `.Design`), **dotnet-ef 10.0.14** e **Dapper 2.1.89**. A connection string de desenvolvimento fica em **User Secrets** (`dotnet user-secrets set ConnectionStrings:PortalTarefas ...`) e a senha do contêiner em `.env`, ambos fora do Git; o repositório traz apenas `.env.example` sem valores.

2. Resumo teórico aplicado ao projeto

DbContext. `PortalTarefasDbContext` é a unidade de trabalho: rastreia as entidades lidas, detecta mudanças e gera os comandos no `SaveChanges`. Seu ciclo de vida é **por requisição** (escopo de DI); não é *thread-safe* e não deve ser compartilhado. Leituras que não serão alteradas usam `AsNoTracking()` para não ocupar o *change tracker*.

Mapeamento. Convenções definem chave (`Id`) e nomes; a **Fluent API** (`TarefaConfiguration`) ajusta limites (`Titulo` 120, `Descricao` 1000), enums gravados como texto, `Prazo` como `date`, valor padrão `SYSDATETIMEOFFSET()`, índice único em `Titulo`, relação 1:N `Tarefa → EventoAuditoria` com exclusão em cascata e a `rowversion`.

Migrations e seed. Migrations versionam a *evolução do esquema* em código revisável; não são backup — não guardam dados nem substituem cópias de segurança, e um `Down` pode apagar colunas com dados. O seed usa `UseSeeding/UseAsyncSeeding` (EF Core 9+), executado no `Migrate()` e no `dotnet ef database update`, e é **idempotente**: só insere se a tabela estiver vazia.

Pooling. `AddDbContextPool` reaproveita instâncias de contexto (menos alocação); por isso o contexto não guarda estado próprio nem recebe serviços com estado no construtor — o pool só reinicia o que o EF conhece. O pool de **conexões** é do provedor ADO.NET (SqlClient) e funciona independentemente: o EF e o Dapper abrem e fecham conexões que voltam ao mesmo pool.

Transações. Cada `SaveChanges` já é atômico (transação implícita). Quando uma operação precisa de **vários** `SaveChanges` (ou SQL manual) como unidade, usa-se `Database.BeginTransactionAsync()`; sem `Commit` o `Dispose` faz rollback. Com `EnableRetryOnFailure`, a transação explícita precisa rodar dentro de `CreateExecutionStrategy().ExecuteAsync` para que a unidade inteira seja repetida. Uma

`DbTransaction` explícita (ADO.NET) é o mesmo mecanismo em nível mais baixo, útil para compartilhar a transação entre EF e Dapper.

Concorrência otimista. Em vez de bloquear, cada linha tem um token; no SQL Server, `rowversion` muda a cada `UPDATE`. O EF gera `UPDATE ... WHERE Id = @id AND Versao = @versaoOriginal`; se nenhuma linha é afetada, lança `DbUpdateConcurrencyException`. Estratégia adotada: **o banco vence** — nada é sobrescrito, o usuário vê os valores atuais e decide se salva de novo com a nova versão.

EF Core × Dapper/ADO.NET. EF: LINQ tipado, rastreamento, migrations e menos código para CRUD. Dapper: SQL escrito à mão, controle total do comando, menor sobrecarga, ideal para consultas de leitura específicas (relatórios, painéis).

3. Código-fonte e evidências

Comandos aplicados (saídas completas em `docs/evidencias/atividade3/`): `dotnet ef migrations add Inicial`, revisão do script com `dotnet ef migrations script --idempotent` e `dotnet ef database update`. Antes: o banco não existia; depois: tabelas `Tarefas`, `EventosAuditoria` e `__EFMigrationsHistory` e 8 tarefas do seed. Trecho do SQL revisado:

```
        [Id] int NOT NULL IDENTITY,
        [Titulo] nvarchar(120) NOT NULL,
        [Descricao] nvarchar(1000) NULL,
        [Situacao] nvarchar(20) NOT NULL,
        [Prioridade] nvarchar(10) NOT NULL,
        [Prazo] date NULL,
        [CriadaEm] datetimeoffset NOT NULL DEFAULT (SYSDATETIMEOFFSET()),
        [Versao] rowversion NOT NULL,
        CONSTRAINT [PK_Tarefas] PRIMARY KEY ([Id])
    );
END;
```

Transação explícita (`TarefaServicoEf.ConcluirAsync`): 1º `SaveChanges` faz o `UPDATE` da tarefa, 2º faz o `INSERT` da auditoria; uma falha simulada entre os dois comandos é lançada para provar o rollback (Tabela 2). **Concorrência:** a edição envia a versão lida em um campo oculto; o serviço a define como `OriginalValue` e o SQL gerado foi:

```
Executed DbCommand (4ms) [Parameters=[@p1='?' (DbType = Int32), @p0='?' (Size = 20), @p2='?' (Size = 8)
(DbType = Binary)], CommandType='Text', CommandTimeout='30'] SET IMPLICIT_TRANSACTIONS OFF; SET NOCOUNT
ON; UPDATE [Tarefas] S …
```

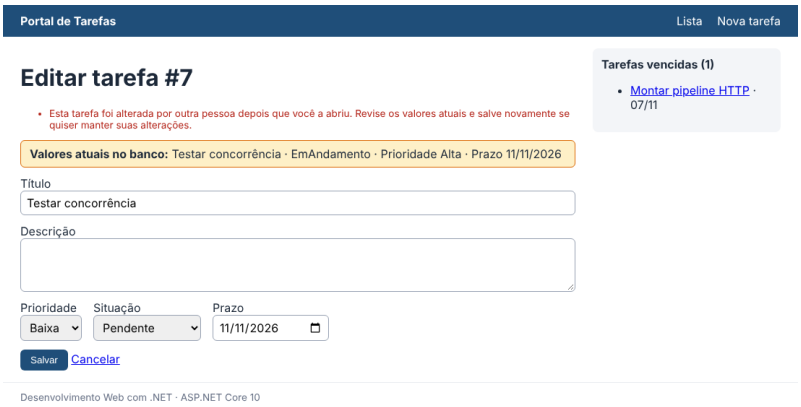


Figura 1 – Segunda sessão recebe o conflito com os valores atuais (nada foi sobrescrito)

Consulta de painel (totais por situação, vencidas e próximo prazo) em duas versões: EF Core com `GroupBy` + projeção + `AsNoTracking`, traduzida em **um único** `SELECT ... GROUP BY` com parâmetro `@hoje`; e Dapper com SQL parametrizado (`@Hoje`, `@Concluida`), sem interpolar valores. Os resultados foram comparados automaticamente (`diff`): **idênticos**.

Tabela 1 – Comparação das duas versões da consulta de painel

Aspecto	EF Core	Dapper
Código	~15 linhas de LINQ, tipado e refatorável	~20 linhas (SQL + mapeamento para record)
Rastreamento	Desligado com `AsNoTracking` e projeção	Não existe (só materializa)
Controle do SQL	Gerado pelo provedor (revisado no log)	Total; mesmo SQL em qualquer versão
Clareza / manutenção	Erros de nome no compilador	Erros de SQL só em execução (coberto por teste)
Uso indicado	CRUD, regras de domínio, transações	Leituras específicas e performance crítica

4. Testes e validações obrigatórias

Tabela 2 – Testes obrigatórios (9 testes de integração em banco SQL Server isolado)

Caso exigido	Resultado obtido	Evidência
Banco a partir das migrations em ambiente limpo	Banco removido e recriado: `Inicial` + `IndiceSituacaoPrazo` aplicadas	`00-banco-limpo.txt`; teste `Migrations_criam_*`
Seed idempotente ao reiniciar	Após 1º e 2º início da aplicação: 8 tarefas (sem duplicar)	`08-seed-idempotente.txt`; `Seed_e_idempotente`
Transação com sucesso e rollback	Falha simulada → 500; tarefa 6 continua `Pendente` e 0 auditorias. Sem falha → 204; `Concluida` + 1 auditoria	`09-transacao.txt`; 2 testes
Conflito com duas edições	Sessão 1 salva; sessão 2 (versão antiga) recebe aviso e valores atuais; versão `…07D7` → `…07DC`	Fig. 1, B1–B3; `10-concorrencia.txt`
Consulta eficiente com projeção	1 `SELECT ... GROUP BY`; `ChangeTracker` vazio após leitura	`12-sql-gerado-painel.txt`; `Leitura_sem_rastreamento_*`

5. Aprimoramentos e análise

- **Índice por migration:** a 2ª migration (`IndiceSituacaoPrazo`) cria `IX_Tarefas_Situacao_Prazo`, que apoia o filtro por situação e a busca de vencidas; o `Down` remove o índice. Evidência: `04-migration-indice.txt`.
- **Teste de integração com banco isolado:** cada classe cria `PortalTarefas_Testes_<guid>` pelas migrations e o remove ao final (`EnsureDeleted`); a conexão vem da variável `PORTAL_SQL_TESTES`.
- **Medição local** (500 execuções após aquecimento, log de SQL desligado): mediana EF 1,28 ms × Dapper 0,94 ms; p95 1,60 × 1,42 ms. Com 8 linhas, na mesma máquina e em Debug, isso **não** permite concluir desempenho geral; só indica a sobrecarga fixa de tradução/materialização do EF.
- **Defeito encontrado depois (unidade 4) e corrigido:** se nenhuma coluna mudava, o EF não emitia `UPDATE` (a `rowversion` é gerada pelo banco) e a versão antiga não era conferida. A

correção marca `Titulo` como modificado, forçando o `WHERE Versao = @original`; o teste de regressão `Versao_antiga_sem_mudancas_gera_conflito` cobre o caso.

Referências

MICROSOFT. **DbContext Lifetime, Configuration, and Initialization**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/dbcontext-configuration/. Acesso em: 9 nov. 2026.

MICROSOFT. **Migrations Overview**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/managing-schemas/migrations/. Acesso em: 9 nov. 2026.

MICROSOFT. **Data Seeding**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/modeling/data-seeding. Acesso em: 9 nov. 2026.

MICROSOFT. **Advanced Performance Topics: DbContext pooling**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/performance/advanced-performance-topics. Acesso em: 9 nov. 2026.

MICROSOFT. **Transactions**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/saving/transactions. Acesso em: 9 nov. 2026.

MICROSOFT. **Handling Concurrency Conflicts**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/saving/concurrency. Acesso em: 9 nov. 2026.

MICROSOFT. **Connection Resiliency**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/ef/core/miscellaneous/connection-resiliency. Acesso em: 9 nov. 2026.

DAPPERLIB. **Dapper**. GitHub, 2025. Disponível em: https://github.com/DapperLib/Dapper. Acesso em: 9 nov. 2026.

Apêndice A – Código-fonte integral

Arquivo: `src/PortalTarefas.Web/Modelos/Tarefa.cs` (commit `614fc20`)

```
using PortalTarefas.Web.Contratos;

namespace PortalTarefas.Web.Modelos;

public enum PrioridadeTarefa { Baixa, Media, Alta }

/// <summary>Modelo de domínio da tarefa. Não é exposto diretamente em views nem na API.</summary>
public class Tarefa
{
    public int Id { get; set; }
    public string Titulo { get; set; } = "";
    public string? Descricao { get; set; }
    public SituacaoTarefa Situacao { get; set; } = SituacaoTarefa.Pendente;
    public PrioridadeTarefa Prioridade { get; set; } = PrioridadeTarefa.Media;
    public DateOnly? Prazo { get; set; }
    public DateTimeOffset CriadaEm { get; set; } = DateTimeOffset.UtcNow;

    /// <summary>Token de concorrência otimista (rowversion no SQL Server).</summary>
    public byte[] Versao { get; set; } = [];

    public List<EventoAuditoria> Auditoria { get; set; } = [];

    public bool EstaVencida(DateOnly hoje) => Prazo < hoje && Situacao != SituacaoTarefa.Concluida;
}

/// <summary>Registro de auditoria gravado na mesma transação da alteração da tarefa.</summary>
public class EventoAuditoria
{
    public long Id { get; set; }
    public int TarefaId { get; set; }
    public string Acao { get; set; } = "";
    public string? Detalhe { get; set; }
    public string? Usuario { get; set; }
    public DateTimeOffset OcorridoEm { get; set; } = DateTimeOffset.UtcNow;
}

/// <summary>Lançada quando outra sessão alterou a tarefa depois de ela ter sido lida.</summary>
public sealed class ConflitoConcorrenciaException(Tarefa? valoresAtuais)
    : Exception("A tarefa foi alterada por outra sessão.")
{
    /// <summary>Valores atuais no banco (null se a tarefa foi excluída).</summary>
    public Tarefa? ValoresAtuais { get; } = valoresAtuais;
}
```

Arquivo: `src/PortalTarefas.Web/Dados/PortalTarefasDbContext.cs` (commit `614fc20`)

```
using Microsoft.EntityFrameworkCore;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Dados;

/// <summary>
/// Unidade de trabalho do EF Core. Registrado com AddDbContextPool: por isso não recebe
/// serviços com estado no construtor nem guarda campos próprios — o pool reinicia apenas
/// o estado conhecido pelo EF (change tracker etc.) ao devolver a instância.
/// </summary>
public class PortalTarefasDbContext(DbContextOptions<PortalTarefasDbContext> options) :
DbContext(options)
{
    public DbSet<Tarefa> Tarefas => Set<Tarefa>();
    public DbSet<EventoAuditoria> Auditoria => Set<EventoAuditoria>();

    protected override void OnModelCreating(ModelBuilder modelBuilder) =>
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(PortalTarefasDbContext).Assembly);
}
```

Arquivo: `src/PortalTarefas.Web/Dados/Configuracoes/TarefaConfiguration.cs` (commit `614fc20`)

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Dados.Configuracoes;

/// <summary>Mapeamento da tarefa com Fluent API (limites, conversões, índices e concorrência).</summary>
public sealed class TarefaConfiguration : IEntityTypeConfiguration<Tarefa>
{
    public void Configure(EntityTypeBuilder<Tarefa> b)
    {
```

```
        b.ToTable("Tarefas");
        b.HasKey(t => t.Id);
        b.Property(t => t.Titulo).IsRequired().HasMaxLength(120);
        b.Property(t => t.Descricao).HasMaxLength(1000);
        // Enums como texto: legível em consultas manuais e estável se a ordem do enum mudar.
        b.Property(t => t.Situacao).HasConversion<string>().HasMaxLength(20).IsRequired();
        b.Property(t => t.Prioridade).HasConversion<string>().HasMaxLength(10).IsRequired();
        b.Property(t => t.Prazo).HasColumnType("date");
        b.Property(t => t.CriadaEm).HasDefaultValueSql("SYSDATETIMEOFFSET()");
        b.Property(t => t.Versao).IsRowVersion();   // rowversion: o SQL Server muda o valor a cada
UPDATE

        b.HasIndex(t => t.Titulo).IsUnique();
        // Índice adicionado na 2ª migration: apoia filtros por situação e a busca de vencidas/painel.
        b.HasIndex(t => new { t.Situacao, t.Prazo }).HasDatabaseName("IX_Tarefas_Situacao_Prazo");

        b.HasMany(t => t.Auditoria).WithOne().HasForeignKey(e =>
e.TarefaId).OnDelete(DeleteBehavior.Cascade);
    }
}

public sealed class EventoAuditoriaConfiguration : IEntityTypeConfiguration<EventoAuditoria>
{
    public void Configure(EntityTypeBuilder<EventoAuditoria> b)
    {
        b.ToTable("EventosAuditoria");
        b.Property(e => e.Acao).IsRequired().HasMaxLength(50);
        b.Property(e => e.Detalhe).HasMaxLength(500);
        b.Property(e => e.Usuario).HasMaxLength(100);
        b.HasIndex(e => new { e.TarefaId, e.OcorridoEm });
    }
}
```

Arquivo: `src/PortalTarefas.Web/Dados/SementeDados.cs` (commit `614fc20`)

```
using Microsoft.EntityFrameworkCore;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Dados;

/// <summary>
/// Seed idempotente: só insere quando a tabela está vazia. É executado pelo EF Core
/// (UseSeeding/UseAsyncSeeding) ao aplicar migrations via Migrate() ou "dotnet ef database update".
/// </summary>
public static class SementeDados
{
    public static async Task SemearAsync(DbContext ctx, CancellationToken ct = default)
    {
        var tarefas = ctx.Set<Tarefa>();
        if (await tarefas.AnyAsync(ct)) return;   // já semeado: reiniciar a aplicação não duplica

        tarefas.AddRange(CriarTarefas(DateOnly.FromDateTime(DateTime.Today)));
        await ctx.SaveChangesAsync(ct);
    }

    public static void Semear(DbContext ctx)
    {
        var tarefas = ctx.Set<Tarefa>();
        if (tarefas.Any()) return;
        tarefas.AddRange(CriarTarefas(DateOnly.FromDateTime(DateTime.Today)));
        ctx.SaveChanges();
    }

    private static IEnumerable<Tarefa> CriarTarefas(DateOnly hoje) =>
    [
        new() { Titulo = "Configurar ambiente .NET 10", Descricao = "Instalar SDK e Visual Studio",
Situacao = SituacaoTarefa.Concluida, Prioridade = PrioridadeTarefa.Alta, Prazo = hoje.AddDays(-10) },
        new() { Titulo = "Montar pipeline HTTP", Descricao = "Middlewares na ordem correta", Situacao =
SituacaoTarefa.EmAndamento, Prioridade = PrioridadeTarefa.Media, Prazo = hoje.AddDays(-2) },
        new() { Titulo = "Escrever relatório 1", Situacao = SituacaoTarefa.Pendente, Prioridade =
PrioridadeTarefa.Baixa, Prazo = hoje.AddDays(7) },
        new() { Titulo = "Criar views Razor", Descricao = "Layout, partial e Tag Helpers", Situacao =
SituacaoTarefa.Concluida, Prioridade = PrioridadeTarefa.Media, Prazo = hoje.AddDays(-5) },
        new() { Titulo = "Mapear entidades no EF Core", Situacao = SituacaoTarefa.EmAndamento, Prioridade
= PrioridadeTarefa.Alta, Prazo = hoje.AddDays(3) },
        new() { Titulo = "Revisar SQL da migration", Situacao = SituacaoTarefa.Pendente, Prioridade =
PrioridadeTarefa.Media, Prazo = hoje.AddDays(-1) },
        new() { Titulo = "Testar concorrência", Situacao = SituacaoTarefa.Pendente, Prioridade =
PrioridadeTarefa.Alta, Prazo = hoje.AddDays(2) },
        new() { Titulo = "Comparar EF Core e Dapper", Situacao = SituacaoTarefa.Pendente, Prioridade =
PrioridadeTarefa.Baixa },
    ];
}
```

Arquivo: `src/PortalTarefas.Web/Dados/Migrations/20261109011900_Inicial.cs` (commit `614fc20`)

```
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace PortalTarefas.Web.Dados.Migrations
{
    /// <inheritdoc />
    public partial class Inicial : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "Tarefas",
                columns: table => new
                {
                    Id = table.Column<int>(type: "int", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    Titulo = table.Column<string>(type: "nvarchar(120)", maxLength: 120, nullable:
false),
                    Descricao = table.Column<string>(type: "nvarchar(1000)", maxLength: 1000, nullable:
true),
                    Situacao = table.Column<string>(type: "nvarchar(20)", maxLength: 20, nullable:
false),
                    Prioridade = table.Column<string>(type: "nvarchar(10)", maxLength: 10, nullable:
false),
                    Prazo = table.Column<DateOnly>(type: "date", nullable: true),
                    CriadaEm = table.Column<DateTimeOffset>(type: "datetimeoffset", nullable: false,
defaultValueSql: "SYSDATETIMEOFFSET()"),
                    Versao = table.Column<byte[]>(type: "rowversion", rowVersion: true, nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Tarefas", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "EventosAuditoria",
                columns: table => new
                {
                    Id = table.Column<long>(type: "bigint", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    TarefaId = table.Column<int>(type: "int", nullable: false),
                    Acao = table.Column<string>(type: "nvarchar(50)", maxLength: 50, nullable: false),
                    Detalhe = table.Column<string>(type: "nvarchar(500)", maxLength: 500, nullable:
true),
                    Usuario = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable:
true),
                    OcorridoEm = table.Column<DateTimeOffset>(type: "datetimeoffset", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_EventosAuditoria", x => x.Id);
                    table.ForeignKey(
                        name: "FK_EventosAuditoria_Tarefas_TarefaId",
                        column: x => x.TarefaId,
                        principalTable: "Tarefas",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                });

            migrationBuilder.CreateIndex(
                name: "IX_EventosAuditoria_TarefaId_OcorridoEm",
                table: "EventosAuditoria",
                columns: new[] { "TarefaId", "OcorridoEm" });

            migrationBuilder.CreateIndex(
                name: "IX_Tarefas_Titulo",
                table: "Tarefas",
                column: "Titulo",
                unique: true);
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.DropTable(
                name: "EventosAuditoria");

            migrationBuilder.DropTable(
```

```
                name: "Tarefas");
        }
    }
}
```

Arquivo: `src/PortalTarefas.Web/Servicos/TarefaServicoEf.cs` (commit `65c1594`)

```
using Microsoft.EntityFrameworkCore;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Dados;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Servicos;

/// <summary>Serviço de tarefas persistido com EF Core (assíncrono, leituras sem rastreamento).</summary>
public sealed class TarefaServicoEf(PortalTarefasDbContext db, ILogger<TarefaServicoEf> logger) :
ITarefaServico
{
    public async Task<IReadOnlyList<Tarefa>> ListarAsync(SituacaoTarefa? situacao, string? busca,
CancellationToken ct = default)
    {
        var consulta = db.Tarefas.AsNoTracking();
        if (situacao is not null) consulta = consulta.Where(t => t.Situacao == situacao);
        if (!string.IsNullOrWhiteSpace(busca)) consulta = consulta.Where(t => t.Titulo.Contains(busca));
        return await consulta.OrderBy(t => t.Id).ToListAsync(ct);
    }

    public Task<Tarefa?> ObterAsync(int id, CancellationToken ct = default) =>
        db.Tarefas.AsNoTracking().FirstOrDefaultAsync(t => t.Id == id, ct);

    public async Task<Tarefa> CriarAsync(Tarefa tarefa, CancellationToken ct = default)
    {
        tarefa.Id = 0;
        tarefa.CriadaEm = DateTimeOffset.UtcNow;
        tarefa.Auditoria.Add(new EventoAuditoria { Acao = "Criada", Detalhe = tarefa.Titulo });
        db.Tarefas.Add(tarefa);
        await db.SaveChangesAsync(ct);   // tarefa + auditoria no mesmo SaveChanges = mesma transação
implícita
        return tarefa;
    }

    public async Task<bool> AtualizarAsync(Tarefa tarefa, string? usuario = null, CancellationToken ct =
default)
    {
        var atual = await db.Tarefas.FirstOrDefaultAsync(t => t.Id == tarefa.Id, ct);
        if (atual is null) return false;

        // A versão enviada pelo cliente passa a ser o "valor original": o UPDATE terá WHERE Versao =
@original.
        db.Entry(atual).Property(t => t.Versao).OriginalValue = tarefa.Versao;
        var mudancas = DescreverMudancas(atual, tarefa);
        atual.Titulo = tarefa.Titulo;
        atual.Descricao = tarefa.Descricao;
        atual.Situacao = tarefa.Situacao;
        atual.Prioridade = tarefa.Prioridade;
        atual.Prazo = tarefa.Prazo;
        // Se nenhum valor de coluna mudou, o EF não emitiria UPDATE (a rowversion é gerada pelo banco e
não
        // entra no SET) e a versão nunca seria conferida. Marcar o título como modificado garante o
        // UPDATE ... WHERE Versao = @original em todos os casos.
        db.Entry(atual).Property(t => t.Titulo).IsModified = true;
        db.Auditoria.Add(new EventoAuditoria { TarefaId = atual.Id, Acao = "Editada", Detalhe = mudancas,
Usuario = usuario });

        try
        {
            await db.SaveChangesAsync(ct);
            tarefa.Versao = atual.Versao;
            return true;
        }
        catch (DbUpdateConcurrencyException ex)
        {
            // Estratégia explícita: o banco vence. Nada é sobrescrito; o usuário recebe os valores
atuais.
            var entrada = ex.Entries.Single(e => e.Entity is Tarefa);
            var valoresBanco = await entrada.GetDatabaseValuesAsync(ct);
            logger.LogWarning("Conflito de concorrência ao editar a tarefa {TarefaId}", tarefa.Id);
            db.ChangeTracker.Clear();
            throw new ConflitoConcorrenciaException((Tarefa?)valoresBanco?.ToObject());
        }
    }

    public async Task<bool> ConcluirAsync(int id, string? usuario, bool simularFalha = false,
CancellationToken ct = default)
```

```
    {
        // Com EnableRetryOnFailure, transações explícitas precisam rodar dentro da estratégia de
execução,
        // para que a unidade inteira (e não só um comando) seja repetida em falhas transitórias.
        var estrategia = db.Database.CreateExecutionStrategy();
        return await estrategia.ExecuteAsync(async () =>
        {
            db.ChangeTracker.Clear();
            await using var transacao = await db.Database.BeginTransactionAsync(ct);

            var tarefa = await db.Tarefas.FirstOrDefaultAsync(t => t.Id == id, ct);
            if (tarefa is null) return false;

            var anterior = tarefa.Situacao;
            tarefa.Situacao = SituacaoTarefa.Concluida;
            await db.SaveChangesAsync(ct);                      // 1º comando: UPDATE Tarefas

            if (simularFalha)
                throw new InvalidOperationException("Falha simulada entre o UPDATE e o registro de
auditoria.");

            db.Auditoria.Add(new EventoAuditoria { TarefaId = id, Acao = "Concluida", Detalhe =
$"{anterior} -> Concluida", Usuario = usuario });
            await db.SaveChangesAsync(ct);                      // 2º comando: INSERT EventosAuditoria

            await transacao.CommitAsync(ct);                    // sem Commit, o Dispose faz rollback
            return true;
        });
    }

    public async Task<bool> ExcluirAsync(int id, CancellationToken ct = default) =>
        await db.Tarefas.Where(t => t.Id == id).ExecuteDeleteAsync(ct) > 0;   // DELETE direto, sem
carregar a entidade

    public Task<bool> ExisteTituloAsync(string titulo, int? ignorarId = null, CancellationToken ct =
default)
    {
        var normalizado = titulo.Trim();
        return db.Tarefas.AnyAsync(t => t.Titulo == normalizado && t.Id != ignorarId, ct);   // collation
CI do SQL Server
    }

    public async Task<IReadOnlyList<Tarefa>> ListarVencidasAsync(DateOnly hoje, CancellationToken ct =
default) =>
        await db.Tarefas.AsNoTracking()
            .Where(t => t.Prazo < hoje && t.Situacao != SituacaoTarefa.Concluida)
            .OrderBy(t => t.Prazo).ToListAsync(ct);

    private static string DescreverMudancas(Tarefa antes, Tarefa depois)
    {
        var partes = new List<string>();
        if (antes.Titulo != depois.Titulo) partes.Add("título");
        if (antes.Descricao != depois.Descricao) partes.Add("descrição");
        if (antes.Situacao != depois.Situacao) partes.Add($"situação {antes.Situacao}-
>{depois.Situacao}");
        if (antes.Prioridade != depois.Prioridade) partes.Add($"prioridade {antes.Prioridade}-
>{depois.Prioridade}");
        if (antes.Prazo != depois.Prazo) partes.Add("prazo");
        return partes.Count == 0 ? "sem alterações" : string.Join(", ", partes);
    }
}
```

Arquivo: `src/PortalTarefas.Web/Servicos/PainelServico.cs` (commit `614fc20`)

```
using Dapper;
using Microsoft.EntityFrameworkCore;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Dados;

namespace PortalTarefas.Web.Servicos;

/// <summary>Linha do painel: totais por situação.</summary>
public sealed record ResumoSituacao(string Situacao, int Quantidade, int Vencidas, DateOnly?
ProximoPrazo);

/// <summary>
/// A mesma consulta de leitura em duas versões: EF Core (LINQ + projeção + AsNoTracking)
/// e Dapper (SQL escrito à mão, com parâmetros).
/// </summary>
public sealed class PainelServico(PortalTarefasDbContext db)
{
    public async Task<IReadOnlyList<ResumoSituacao>> ResumoEfAsync(DateOnly hoje, CancellationToken ct =
default)
    {
```

```
        // Projeção traduzida para um único SELECT ... GROUP BY; só as colunas agregadas trafegam.
        var linhas = await db.Tarefas.AsNoTracking()
            .GroupBy(t => t.Situacao)
            .Select(g => new
            {
                Situacao = g.Key,
                Quantidade = g.Count(),
                Vencidas = g.Count(t => t.Prazo < hoje && t.Situacao != SituacaoTarefa.Concluida),
                ProximoPrazo = g.Where(t => t.Prazo >= hoje).Min(t => t.Prazo)
            })
            .ToListAsync(ct);
        return linhas
            .Select(l => new ResumoSituacao(l.Situacao.ToString(), l.Quantidade, l.Vencidas,
l.ProximoPrazo))
            .OrderBy(r => r.Situacao, StringComparer.Ordinal)
            .ToList();
    }

    public async Task<IReadOnlyList<ResumoSituacao>> ResumoDapperAsync(DateOnly hoje, CancellationToken
ct = default)
    {
        const string sql = """
            SELECT  Situacao,
                    COUNT(*) AS Quantidade,
                    SUM(CASE WHEN Prazo < @Hoje AND Situacao <> @Concluida THEN 1 ELSE 0 END) AS
Vencidas,
                    MIN(CASE WHEN Prazo >= @Hoje THEN Prazo END) AS ProximoPrazo
            FROM    dbo.Tarefas
            GROUP BY Situacao
            ORDER BY Situacao;
            """;
        // Reaproveita a conexão do DbContext (mesmo pool de conexões); valores sempre via parâmetros.
        var conexao = db.Database.GetDbConnection();
        var linhas = await conexao.QueryAsync<LinhaPainel>(new CommandDefinition(sql,
            new { Hoje = hoje.ToDateTime(TimeOnly.MinValue), Concluida = nameof(SituacaoTarefa.Concluida)
},
            cancellationToken: ct));
        return linhas.Select(l => new ResumoSituacao(l.Situacao, l.Quantidade, l.Vencidas,
            l.ProximoPrazo is { } d ? DateOnly.FromDateTime(d) : null)).ToList();
    }

    private sealed record LinhaPainel(string Situacao, int Quantidade, int Vencidas, DateTime?
ProximoPrazo);
}
```

Registro no `Program.cs` (commit `614fc20`):

```
var conexao = builder.Configuration.GetConnectionString("PortalTarefas")
    ?? throw new InvalidOperationException("Connection string 'PortalTarefas' não configurada. ...");
builder.Services.AddDbContextPool<PortalTarefasDbContext>(o => o
    .UseSqlServer(conexao, sql => sql.EnableRetryOnFailure(maxRetryCount: 3))
    .UseSeeding((ctx, _) => SementeDados.Semear(ctx))
    .UseAsyncSeeding((ctx, _, ct) => SementeDados.SemearAsync(ctx, ct)),
    poolSize: 128);
builder.Services.AddScoped<ITarefaServico, TarefaServicoEf>();
builder.Services.AddScoped<PainelServico>();
```

Apêndice B – Evidências de execução

Transação: rollback e commit (antes/depois no banco)

```
== Transação explícita: concluir tarefa 6 + registrar auditoria ==
-- ANTES
Id Situacao
-- --------
6 Pendente
EventosDaTarefa6
----------------
0
### Rollback provocado (falha entre o UPDATE e o INSERT de auditoria)
$ curl -X POST https://localhost:7080/api/dev/tarefas/6/concluir?simularFalha=true
HTTP/2 500
content-type: application/problem+json
cache-control: no-cache,no-store
x-correlation-id: f6c5a537c09b47578924369dfcbced3b
{
    "type": "https://datatracker.ietf.org/doc/html/rfc9110#section-15.6.1",
    "title": "Erro interno ao processar a requisição.",
    "status": 500,
    "detail": "Informe o identificador de correlação ao suporte.",
    "instance": "POST /api/dev/tarefas/6/concluir",
    "traceId": "00-b060d669dbc6a8838dd215a855e34d88-8e854d53353a6e9d-00",
    "correlationId": "f6c5a537c09b47578924369dfcbced3b"
```

```
}
-- DEPOIS DO ROLLBACK (nada mudou)
Id Situacao
-- --------
6 Pendente
EventosDaTarefa6
----------------
0
### Operação com sucesso (commit)
$ curl -X POST https://localhost:7080/api/dev/tarefas/6/concluir
HTTP/2 204
x-correlation-id: ad912fd15ba34a3a8d87ab44a7f8e4c4
-- DEPOIS DO COMMIT (tarefa e auditoria gravadas juntas)
Id Situacao
-- --------
6 Concluida
TarefaId Acao Detalhe Usuario
-------- ---- ------- -------
6 Concluida Pendente -> Concluida dev
```

Concorrência com duas sessões

```
== Concorrência otimista (rowversion) com duas sessões de navegador ==
-- ANTES
Id Titulo Situacao Prioridade Versao
-- ------ -------- ---------- ------
7 Testar concorrência Pendente Alta 0x00000000000007D7
versão vista pelas duas sessões: AAAAAAAAB9c= AAAAAAAAB9c=
sessão 2 após conflito - nova versão no formulário: AAAAAAAAB9s=
-- DEPOIS (a alteração da sessão 1 foi preservada; a sessão 2 salvou só após revisar os valores atuais)
Id Titulo Situacao Prioridade Versao
-- ------ -------- ---------- ------
7 Testar concorrência EmAndamento Baixa 0x00000000000007DC
Acao Detalhe
---- -------
Editada situação Pendente->EmAndamento
Editada prioridade Alta->Baixa
```

Seed idempotente e SQL gerado pelo EF para o painel

```
== Seed idempotente: a aplicação é iniciada duas vezes (Migrate + UseAsyncSeeding a cada início) ==
-- após o início 1:
TotalTarefas
------------
8
-- após o início 2:
TotalTarefas
------------
8


Executed DbCommand (3ms) [Parameters=[@hoje='?' (DbType = Date)], CommandType='Text',
CommandTimeout='30'] SELECT [t].[Situacao], COUNT(*) AS [Quantidade], COUNT(CASE     WHEN [t].[Prazo] <
@hoje AND [t].[Situacao] <> N'Concluida' THEN 1 END) AS [Vencidas], MIN(CASE     WHEN [t].[Prazo] >=
@hoje THEN [t].[Prazo] END) AS [ProximoPrazo] FROM [Tarefas] AS [t] GROUP BY [t].[Situacao]
```

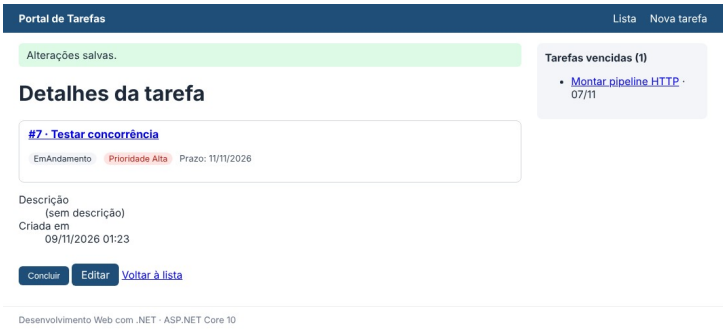


Figura B1 – Sessão 1 salvou a alteração

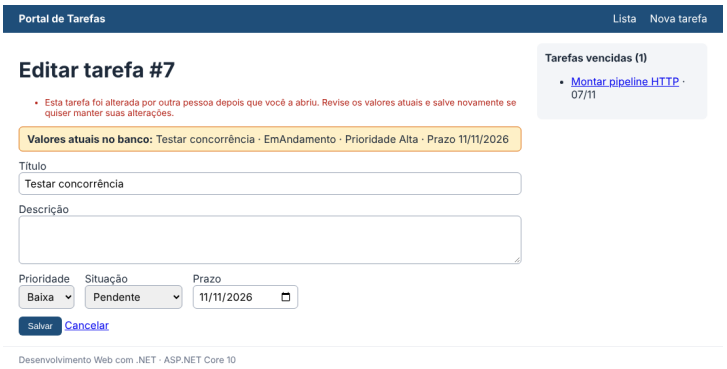


Figura B2 – Sessão 2 com versão antiga: conflito detectado

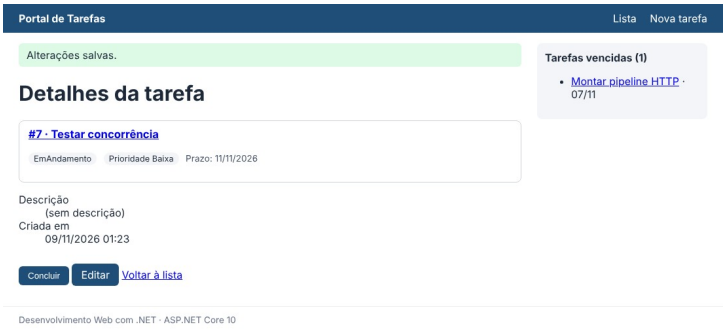


Figura B3 – Sessão 2 salvou após revisar os valores atuais